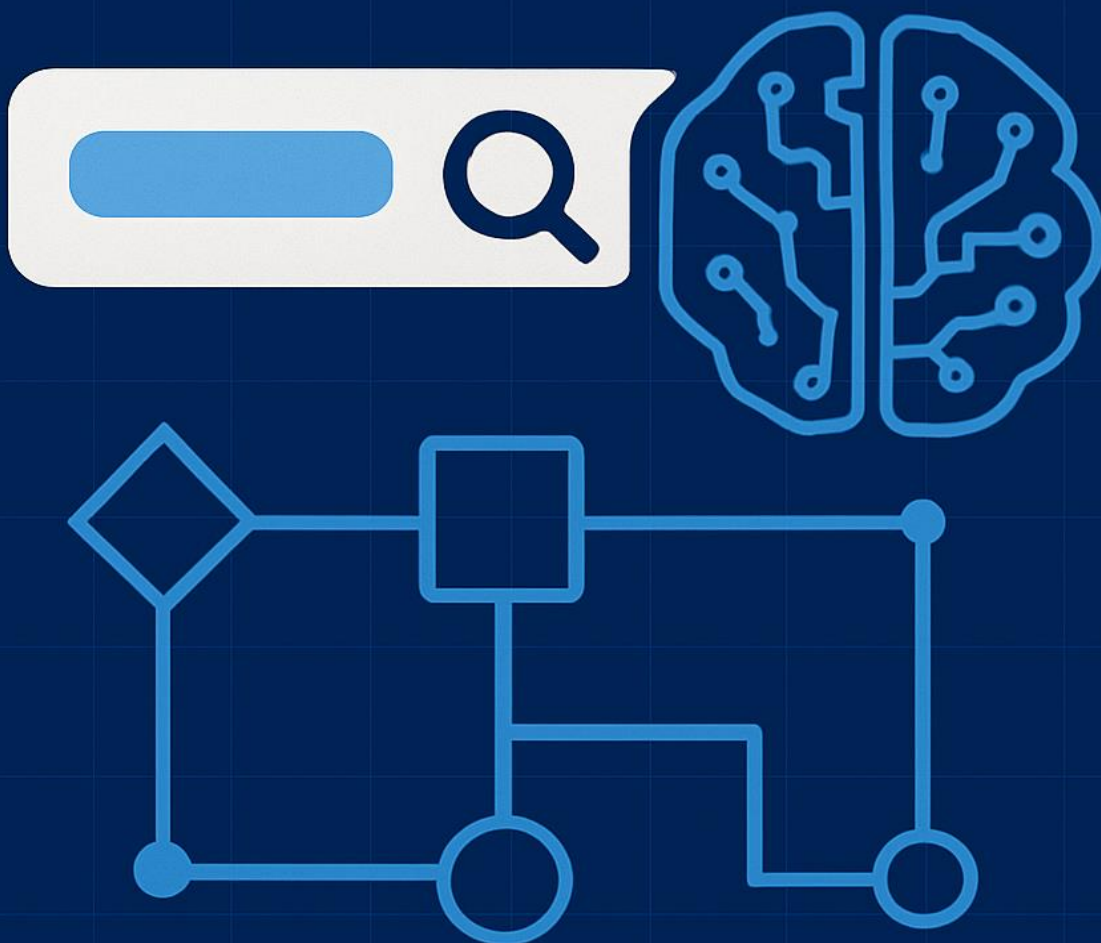




SEARCH ALGORITHMS in AI





Team Members



- Amr Abdelftah Mahmoud Abdelmonem
- Mohamed Ibrahim Al-Arabi Taha
- Mustafa Mahmoud Ibrahim Shahata
- Jana Mahmoud Mohamed Elhadidy
- Ayatullah Mohamed Abdelhamid
- Hoor Amir Essa
- Layla Khaled Mohamed Salama
- Farah El-Said Sobhy Mahmoud
- Omar Hussein Mohamed Ali
- Yomna Ayman Ahmed Mohamed

Course: ELE 161

Search Algorithms in AI

Lecturer: Prof. Ahmed Magdy

AIEP - Second year

2025-11-24



Table of Contents



Page	Title	
3	Introduction	
4	Prior Solutions	
7	Solution Requirements	
10	Algorithm Strengths & Weaknesses	
14	Results	
20	Algorithm Comparison	
22	Conclusion	
23	References	

1. Introduction



This report will cover the process of building an application that uses a GUI to present and demonstrate how different types of searching algorithms works to a user.

With the help of Visuals that the user can interact with.

This report also mentions prior applications that serves the same purpose and the difference between them and the one we are designing.

This report will also include a lot of different topics starting with...

2. Prior Solutions



The application we are working on is not an original idea.

It is an idea that many people before us tried to do, so in this section of the report we will review some of the applications that were made before ours and highlight their pros & cons and show how our application differs from them.

1. VisuAlgo

VisuAlgo is a well-known educational platform designed by Associate Professor Steven Halim (NUS) in 2011 to visualize data structures and algorithms.

It includes demonstrations of graph search methods, priority queues, and pathfinding techniques.

Pros of VisuAlgo:

- So simple and easy to use (*Figure 1*)
- Fast implementation
- Works well on any platform

The popular VisuAlgo was a great application for its time but as mentioned above due to it being an old website it faced some issues.

Cons of VisuAlgo:

- Limited to visualizing
- No interactive method of learning



ASSOCIATE
PROFESSOR
STEVEN HALIM

The designer of the well-known website ViusAlgo.

The Indonesian professor is a computer scientist and educator known for his significant contributions to competitive programming and algorithmic education. He is a long-time faculty member at the School of Computing, National University of Singapore (NUS), where he teaches algorithms, data structures, and problem-solving at both undergraduate and graduate levels.

Completed his **PhD in Computer Science** at NUS between **2004 and 2010** and designed the popular website **VisuAlgo** all alone at the age of **29**.



(Figure 1)-(VisuAlgo GUI)

2. Online Pathfinding Visualizers

There are many online pathfinding visualizers available on the web that focus on demonstrating algorithms such as Breadth-First Search (BFS) & Depth-First Search (DFS).

Pros of online pathfinding Visualizers:

- Easy to access
- Fast implementation
- Interactive GUI (Figure 2)



(Figure 2)

These tools usually provide a grid-based interface where users can place obstacles and observe how each algorithm searches for a path.

Although these visualizers effectively show the basic behavior of search algorithms, they often lack deeper insight into internal operations.

Cons of online pathfinding Visualizers:

- The lack of insight into internal operations such as the explored nodes or cost calculations
- Confusing with how Every website just have few search algorithms
- May cost a subscription.
- contains all the major searching methods.

These prior solutions provide visualizations but they share several limitations:

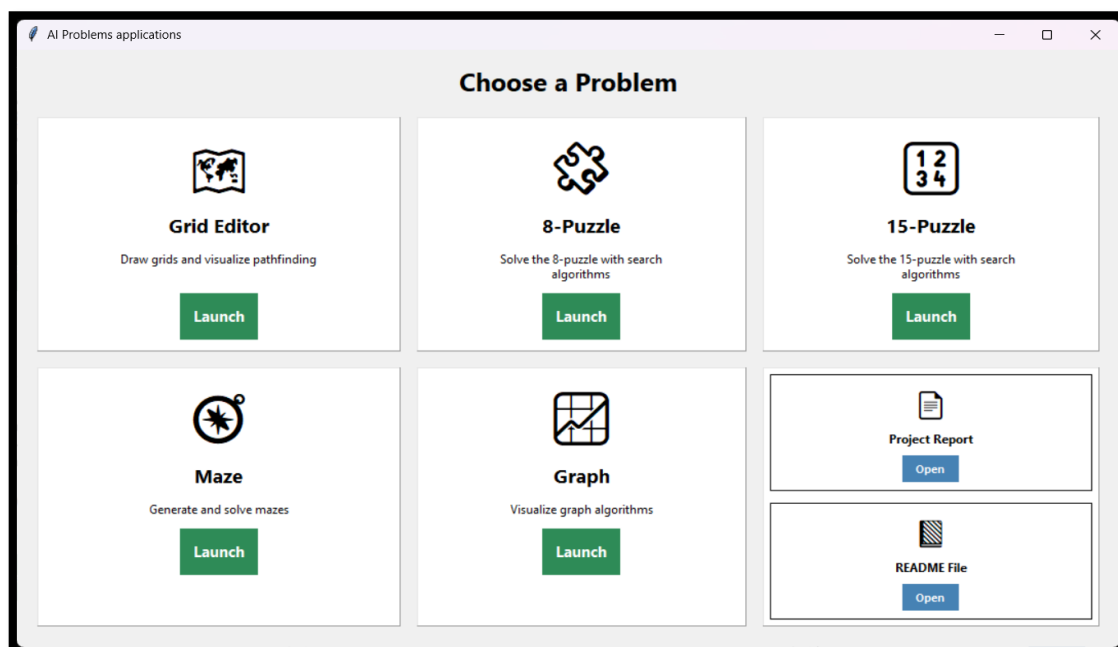
- Limited customization of the search environment.
- Lack of detailed algorithm metrics (cost, time, explored nodes).
- Each application have only 2-3 searching methods.
- Limited insight into internal data structures used during search.
- Not designed as a unified educational tool for algorithm comparison that

How our application differs from all the applications above ?

Our application aims to serve the initial purpose while solving the problem in the already existing application by providing:

- A fully interactive GUI
- The ability to customize obstacles, start and goal positions
- The ability to choose between a lot of searching methods
- Visualization of internal algorithm components (explored set, cost)
- Direct comparison of performance across multiple search algorithms
- A user-friendly educational tool suitable for both beginners and students

By providing these into our application we would overcome the problems that were in the prior applications.



(Photo of our GUI main menu)

3. Solution Requirements



To design this effective and educational visualization tool for classical search algorithms the application must meet a set of requirements.

These requirements ensure that the application is functional, interactive, and capable of showing the behavior of different search techniques.

The requirements can be divided into two main categories:

Functional and Non-Functional:

Functional

The functional requirements are just the features that the application offers and they are:

1. Algorithm Visualization

The application must be able to visualize the execution of multiple classical search algorithms such as:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search
- Greedy Best-First Search
- A* Search

And the list goes on...

Each of these searching algorithms must be visualized step by step using animated tools.

2. Interactive User Interface

The system must include a graphical user interface (GUI) that allows users to:

- Select an algorithm
- Set start and goal positions
- Add or remove obstacles
- Configure grid size or environment
- Start, pause, or step through the search process

3. Real-Time Feedback

During algorithm execution, the tool must provide visual indicators such as:

- Explored nodes or closed list
- Frontier or open list
- Current path
- Final solution path

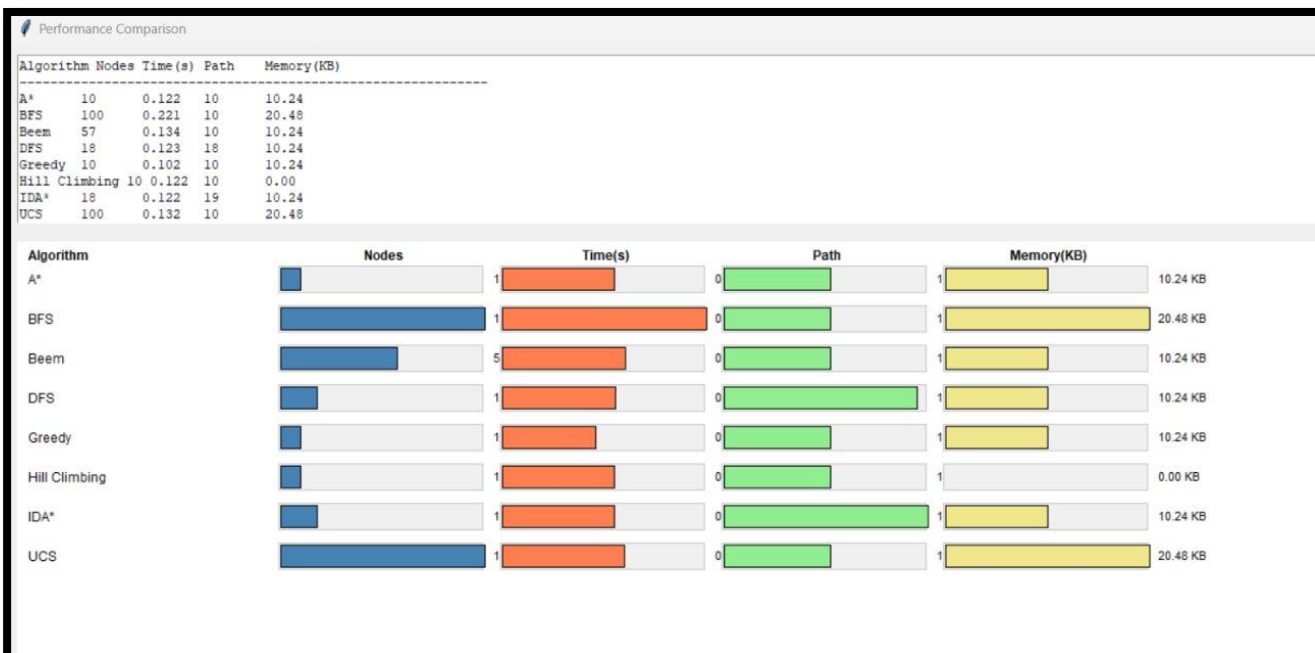
This ensures the user understands how each algorithm explores the environment.

4. Performance Metrics

The application must compute and display quantitative metrics that help compare algorithms, including:

- Number of expanded nodes
- Path cost
- Execution time
- Optimality of the found path

Considering these metrics is the most efficient way to determine whether the application works well or not



(Photo of performance comparison in our GUI)

Non-Functional

Non-functional requirements define constraints and quality attributes of the system.

1. Usability

The GUI must be simple and easy to use for students and beginners.
Controls should be clearly labeled and accessible.

2. Performance

The application must run smoothly with minimal lag during visualization.
Animations should be clear and responsive even when large grids are used.

3. Modularity

The system architecture should be modular allowing additional algorithms or features to be added easily in the future without major redesign.

4. Reliability

The tool should behave consistently and handle invalid inputs gracefully.
Errors or crashes must be minimized.

5. Visual Clarity

Colors, animations, and markers must be used in a way that clearly distinguishes different states such as:

- Explored
- Obstacles
- Final path

These requirements form our imagination of the final form of our application and by working on achieving them we will end up with an application that will serve as a comprehensive educational tool that helps users understand how classical search algorithms operate and how they differ in terms of performance and behavior.

4. Algorithms Strengths & Weaknesses



Classical search algorithms differ significantly in their behavior and efficiency for different types of problems.

Understanding their strengths and weaknesses is essential for analyzing performance and selecting the most appropriate algorithm for a given task.

Starting with (BFS) now we will cover the strengths and weaknesses of the search algorithms we will use in our application:

1. Breadth-First Search (BFS)

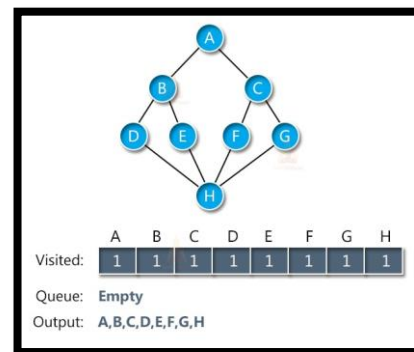
(BFS) is a graph traversal algorithm that explores a graph or tree level by level starting from a source node. *(As shown in Figure 3)*

Strengths:

- Complete for finite graphs
- Optimal when all step costs are equal
- Behavior is predictable and easy to understand

Weaknesses:

- Very high memory consumption
- Slow in deep or large search spaces
- Inefficient for weighted problems



(Figure 3)

2. Depth-First Search (DFS)

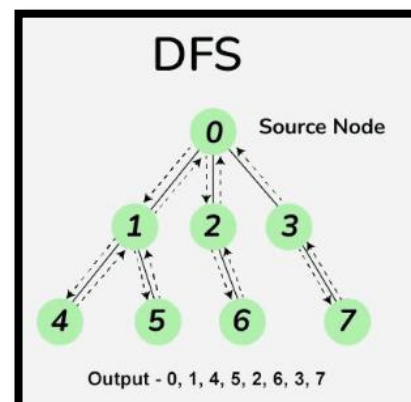
Algorithm for traversing or searching a graph or tree data structure by going as deep as possible along each branch before backtracking. *(As shown in Figure 4)*

Strengths:

- Low memory usage
- Can quickly find deep solutions
- Simple implementation

Weaknesses:

- Not complete in infinite spaces
- Not optimal
- Can get stuck exploring deep irrelevant paths



(Figure 4)

3. Uniform Cost Search (UCS)

Uninformed search algorithm used in Artificial Intelligence to find the least-cost path from a starting node to a goal node in a weighted graph.

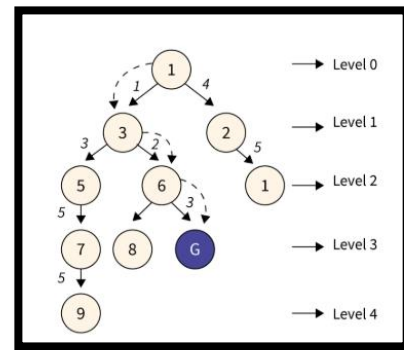
Unlike other search algorithms that might prioritize depth or breadth, UCS prioritizes nodes based on the cumulative cost incurred to reach them from the starting node. *(As shown in figure 5)*

Strengths:

- Always finds the lowest-cost path with positive edge costs
- Complete
- Ideal for weighted graphs

Weaknesses:

- Slower than BFS when costs vary
- High memory usage
- Priority queue operations increase complexity



(Figure 5)

4. Greedy Best-First Search

Informed search algorithm used in Artificial Intelligence to find paths from a starting state to a goal state.

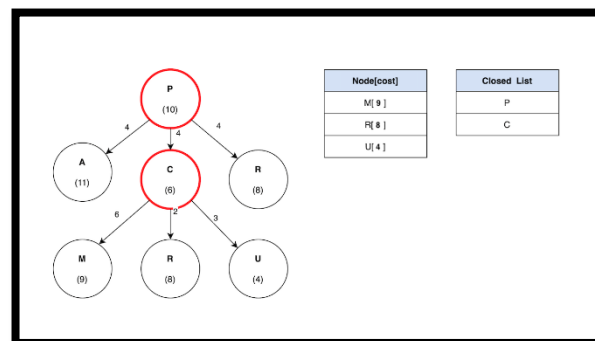
It operates by always expanding the node that appears to be closest to the goal according to a heuristic function. *(As shown in figure 6)*

Strengths:

- Very fast in many practical cases
- Strong heuristic guidance toward the goal
- Good for large open search spaces

Weaknesses:

- Not optimal
- Not always complete
- Completely depends on heuristic quality



(Figure 6)

5. A* Search

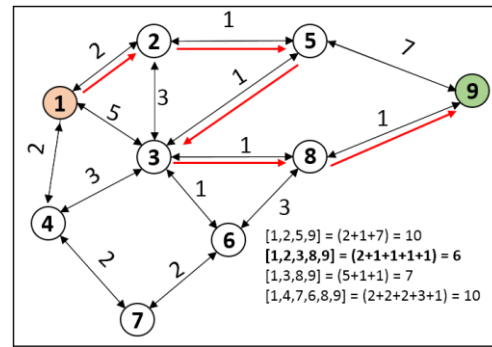
A pathfinding and graph traversal algorithm that finds the shortest path between a start and goal node. (As shown in figure 7)

Strengths:

- Optimal with admissible heuristics
- Efficient combination of cost + heuristic
- Performs well in a wide range of domains

Weaknesses:

- Memory heavy
- Performance drops with weak heuristics
- More complex to design and tune



(Figure 7)

6. Beam Search

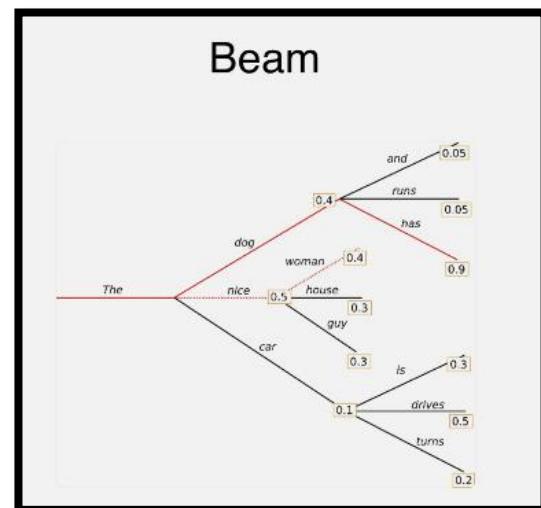
A heuristic search algorithm used in machine learning and natural language processing to find an approximate solution to a problem when the number of possibilities is too large for an exhaustive search. (As shown in figure 8)

Strengths:

- Much lower memory usage than A* or BFS
- Very fast for large spaces
- Focuses only on the most promising nodes

Weaknesses:

- Not complete
- Not optimal
- Can easily discard the correct path if the beam width is too small
- Strongly dependent on heuristic accuracy



(Figure 8)

7. Iterative Deepening A* (IDA*)

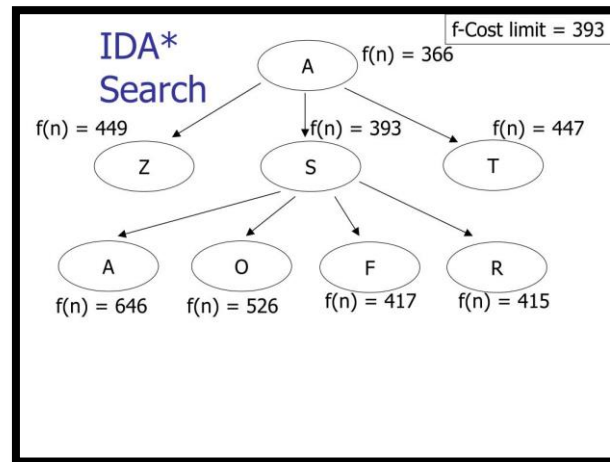
A graph traversal and pathfinding algorithm that combines the benefits of Iterative Deepening Depth-First Search and the A* search algorithm to find the shortest path. (As shown in figure 9)

Strengths:

- Lower memory usage than A*
- Complete and optimal with admissible heuristics
- Ideal for large state spaces (puzzles like the 15-Puzzle in our application)

Weaknesses:

- Repeated exploration increases time cost
- Performance depends heavily on heuristic quality
- Worst-case runtime can be large



(Figure 9)

8. Hill Climbing Search

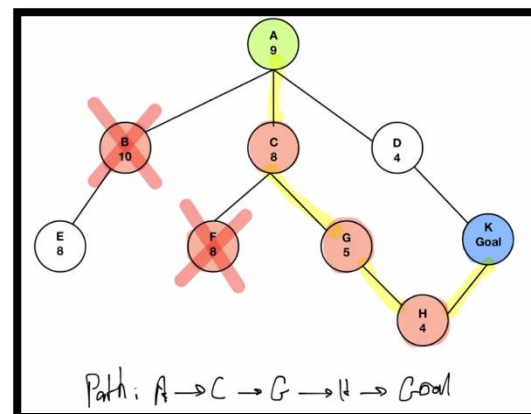
An optimization algorithm in computer science that iteratively improves a solution by moving it to a neighboring state with a better value like a climber ascending a hill. (As shown in figure 10)

Strengths:

- Very simple and fast
- Requires minimal memory
- Moves directly toward improving states

Weaknesses:

- Gets stuck in:
- Not complete
- Not optimal
- Requires strategies like random restarts or simulated annealing to improve reliability



(Figure 10)

5. Results



In this project, we compared several search algorithms on different AI problems.

The algorithm evaluated are:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform-Cost Search (UCS)
- Greedy Best-First Search (Greedy)
- A* Search (A*)
- Iterative Deepening A* (IDA*)
- Hill Climbing
- Beam Search

These algorithms were tested on the following problems:

- Grid pathfinding
- Maze pathfinding
- 8-Puzzle
- 15-Puzzle
- Graph search on the Romania Map (Arad → Lugoj)

For each algorithm–problem combination, we collected the following performance metrics:

- **Time (s):** Execution time in seconds.
- **Nodes Expanded / Nodes:** Number of expanded nodes during the search.
- **Path Length:** Number of steps/moves in the final solution path.
- **Memory (KB):** Memory consumption in kilobytes when available.
- **Solved:** Whether the algorithm successfully found a solution

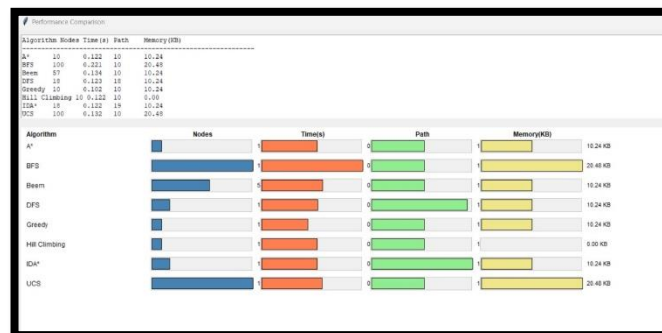
All experiments were run using the provided GUI, with the same initial and goal states for each problem to ensure a fair comparison.

Starting with Grid Pathfinding...

Algorithm	Nodes Expanded	Path Length	Time (s)
A*	10	10	0.122
BFS	100	10	0.221
DFS	18	18	0.123
Greedy	10	10	0.102
UCS	100	10	0.132
Hill	10	10	0.122
IDA*	18	19	0.122
Beam	57	10	0.134

Best configuration reported for the grid:

- **Best algorithm:** A*
- **Nodes Expanded:** 10
- **Path Length:** 10
- **Time:** 0.122 s
- **Memory:** 10.24 KB



Analysis

(Grid performance comparison)

Several algorithms (A*, Greedy, Hill Climbing and Beam Search) achieved an optimal path of length 10. However, **A*** was selected as the best algorithm for this problem because it combines:

- Short path (10 steps)
- Very small number of expanded nodes (10)
- Low execution time (0.122 s)
- Low memory usage (≈ 10.24 KB)

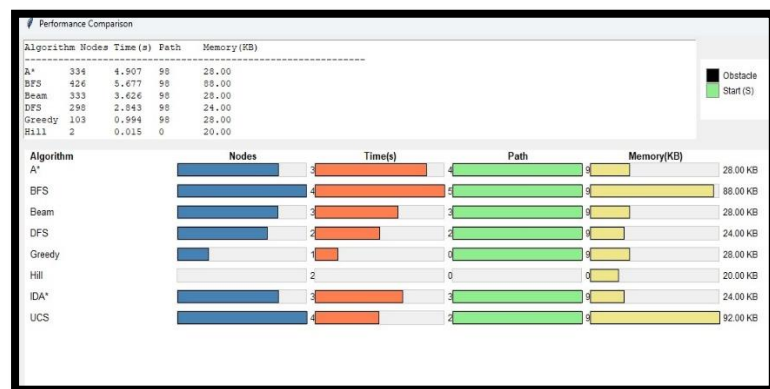
In contrast, **BFS** and **UCS** require expanding 100 nodes to obtain the same path length, which demonstrates that uninformed search explores many more states than informed algorithms that use heuristics

2) Maze Pathfinding

Algorithm	Time (s)	Memory (KB)	Nodes Expanded	Path Length	Solved
A*	4.9067	28.00	334	98	✓
BFS	5.6766	88.00	426	98	✓
Beam	3.6258	28.00	333	98	✓
DFS	2.8434	24.00	298	98	✓
Greedy	0.9943	28.00	103	98	✓
Hill	0.0155	20.00	2	0	✗
IDA*	3.8466	24.00	334	98	✓
UCS	2.7934	92.00	428	98	✓

Analysis

For the maze problem, almost all algorithms except Hill Climbing successfully found a path of length 98. **Greedy** achieved the lowest execution time (0.9943 s) and expanded only 103 nodes, which is significantly fewer than BFS and UCS (426 and 428 nodes respectively) for the same path length.



(Maze performance comparison)

This clearly shows the advantage of heuristic-based algorithms in large state spaces. Hill Climbing failed to solve the problem (path length 0 and only 2 expanded nodes), which indicates that it got stuck in a local optimum or dead end. Even though Greedy is the fastest here, **A*** can still be considered more reliable in general because it is designed to find optimal or near-optimal paths, whereas Greedy may fail or return sub-optimal solutions in other maze configurations.

3) 8-Puzzle

Algorithm	Solved	Nodes Expanded	Path Length	Time (s)	Memory
A*	✓	889	21	3.2765	N/A
BFS	X	—	—	—	—
DFS	X	—	—	—	—
Greedy	✓	76	37	4.8963	N/A
UCS	X	—	—	—	—
Hill	X	—	—	—	—
IDA*	✓	1104	21	2.2701	N/A
Beam	X	—	—	—	—

The best algorithm for this problem is: **IDA***

Analysis

For the 8-Puzzle problem, only three algorithms managed to find a solution: **A***, **Greedy**, and **IDA***.

- **A*** and **IDA*** both achieved relatively short solution paths of length 21.
- **IDA*** reached this path with 1104 expanded nodes and the fastest runtime (2.2701 s).
- **A*** expanded 889 nodes and required 3.2765 s.
- **Greedy** expanded the fewest nodes (76), but produced a significantly longer path (37 moves) and took 4.8963 s.

Although the project's raw output states that the *best* algorithm is **Greedy**, this comes with a trade-off: Greedy explores far fewer nodes but does not guarantee an optimal solution. In contrast, A* and IDA* generate much shorter and higher-quality solution paths at the cost of expanding more nodes.

4) 15-Puzzle

Algorithm	Solved
A*	X
BFS	X
DFS	X
Greedy	X
UCS	X
Hill	X
IDA*	X
Beam	X

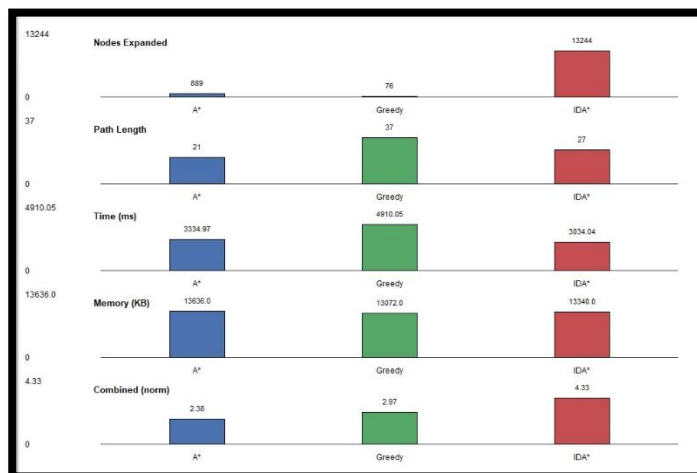
(8-Puzzle performance comparison)

Analysis

For the 15-Puzzle problem, none of the algorithms were able to find a solution under the current settings.

This is expected, because the 15-Puzzle has a very large state space and can require a huge number of expansions and a long running time.

The failure of all algorithms in this configuration indicates that more advanced heuristics, better pruning techniques, or stricter time limits would be needed to handle this problem effectively.



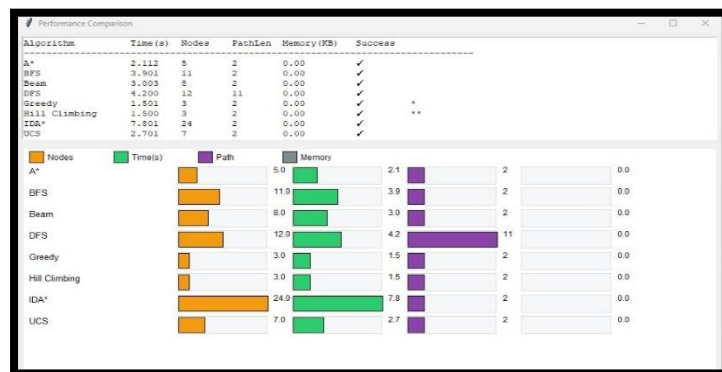
5) Graph Search (Romania Map: Arad → Lugoj)

Algorithm	Time (s)	Mem (KB)	Nodes	PathLen	Solved
A*	2.1123	0.00	5	2	✓
BFS	3.9009	0.00	11	2	✓
Beam	3.0028	0.00	8	2	✓
DFS	4.2001	0.00	12	11	✓
Greedy	1.5006	0.00	3	2	✓
Hill Climbing	1.5004	0.00	3	2	✓
IDA*	7.8006	0.00	24	2	✓
UCS	2.7005	0.00	7	2	✓

Analysis

In the Romania map problem, all algorithms successfully found a path from Arad to Lugoj.

Most of them produced a path with the same length (2 edges), except for **DFS**, which returned a much longer path of length 11 due to its depth-first nature and lack of guidance.



(Graph performance comparison)

From a performance point of view:

- **Greedy** and **Hill Climbing** achieved the best results in terms of runtime and number of expanded nodes (3 nodes in about 1.5 s).
- **A*** and **UCS** also performed well, expanding only 5 and 7 nodes respectively.
- **IDA*** was clearly the slowest (7.8006 s) and expanded the largest number of nodes (24) in this experiment. These results again highlight that heuristic-guided algorithms are very efficient when a good heuristic is available.

6. Algorithm Comparison



Across all experiments, the performance of the algorithms varied significantly depending on the problem type.

- **Uninformed search (BFS, DFS, UCS)**

- BFS and UCS often find optimal or near-optimal paths, but they typically expand a large number of nodes (e.g., 100 nodes in the grid problem).
- DFS can be fast in some cases, but it can also produce very long or poor-quality paths (e.g., path length 11 in the graph problem) and may fail on more complex puzzles.

- **Heuristic-based search (Greedy, A*, IDA*)**

- These algorithms use heuristic information to guide the search, which generally leads to fewer expanded nodes and shorter runtimes.
- **A*** usually offers a good balance between path optimality and efficiency.
- **Greedy** is often the fastest and expands very few nodes (e.g., 103 nodes in the maze, 3 nodes in the graph), but it does not guarantee optimal solutions and may fail in some hard configurations.
- **IDA*** can achieve optimal paths like A*, but sometimes at the cost of higher runtime and more node expansions (e.g., 1104 nodes in the 8-Puzzle).

- **Local search (Hill Climbing, Beam Search)**

- Hill Climbing can be extremely fast but is very sensitive to local optima and can easily fail, as seen in the maze and some puzzle problems.
- Beam Search shows intermediate behaviour: it is faster than exhaustive search methods but less reliable than A* or Greedy in terms of solution quality.

The Best Algorithm:

Choosing the “best” algorithm depends on what we care about most:

- If **solution quality** (shortest path) is the main objective, algorithms like **A*** and **IDA*** are preferred.
- If **speed** and **low memory usage** are more important, **Greedy** and sometimes **Hill Climbing** or **Beam Search** can be more attractive.
- For very large state spaces (such as the 15-Puzzle), all tested algorithms struggled under the current configuration, which suggests the need for more advanced heuristics or different search strategies.

Based on the overall experiments in this project, **A*** can be considered the most generally suitable algorithm because:

- It consistently finds optimal or near-optimal solutions in the grid, maze, graph and 8-Puzzle problems.
- It uses significantly fewer nodes than uninformed strategies like BFS and UCS.
- Its runtime is acceptable in most scenarios.

However, the results also show that **Greedy search** can be an excellent choice when extremely fast responses are required and slightly sub-optimal solutions are acceptable, especially in the maze and graph problems where it expanded very few nodes with very low execution time.

7. Conclusion



This project presented the design, implementation, and evaluation of an interactive GUI-based application for visualizing classical search algorithms in Artificial Intelligence. The main objective was to provide an educational tool that allows users to observe, compare, and understand how different search strategies behave across various problem domains.

Through a series of experiments on grid pathfinding, maze navigation, puzzle problems (8-Puzzle and 15-Puzzle), and graph search using the Romania map, the strengths and limitations of multiple algorithms were clearly demonstrated. The results showed that uninformed search methods such as Breadth-First Search (BFS) and Uniform Cost Search (UCS) are reliable but often inefficient due to the large number of expanded nodes and higher memory consumption. Depth-First Search (DFS), while memory-efficient, proved to be unreliable and sometimes produced poor-quality solutions.

In contrast, heuristic-based algorithms significantly improved performance. A* consistently achieved a strong balance between optimal solution quality and efficiency, making it the most generally effective algorithm across different problem types. Greedy Best-First Search demonstrated excellent speed and low node expansion in many scenarios, although it sometimes produced suboptimal paths. Iterative Deepening A* (IDA*) showed promise for puzzle problems by achieving optimal paths with reduced memory usage, albeit at the cost of increased runtime.

Local search methods such as Hill Climbing and Beam Search were extremely fast in some cases but lacked reliability due to their sensitivity to local optima and heuristic accuracy. The failure of all tested algorithms to solve the 15-Puzzle under the given configuration highlighted the complexity of very large state spaces and emphasized the importance of advanced heuristics and optimization techniques.

Overall, the developed application successfully fulfilled its functional and non-functional requirements by offering a fully interactive visualization environment, detailed performance metrics, and direct algorithm comparisons. The tool serves as a valuable educational resource for students and beginners learning search algorithms in AI. Future improvements could include the integration of more advanced heuristics, additional algorithms, and optimization strategies to handle larger and more complex problem domains.

8. References



In this our research and procces of making this project and report we used the help of these references:

- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson Education.
- Halim, S. (2011). *VisuAlgo – Visualising Data Structures and Algorithms*. National University of Singapore.
<https://visualgo.net>
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Nilsson, N. J. (1980). *Principles of Artificial Intelligence*. Morgan Kaufmann.
- Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.

Team Path Seeker



Pathfinder AI Project

Lecturer: Prof. Ahmed Magdy

AIEP - Second year

New Ismailia National University